Figure 1.13 High-level application structure

The `Supervisor` watches over the components and takes action when they crash. For example, it can decide to continue running when the `Mentions` component or the `TeamFinder` doesn't work. If both `Conversation` and `NotifyUser` stop working, the `Supervisor` can decide to restart or stop the application since there's no reason to continue. A component can send a message to the `Supervisor` when it fails, and the `Supervisor` can send a message to a component to stop or try to restart. As you'll see, this is conceptually how Akka provides error recovery, as discussed in chapter 4.

In the next section, you'll learn first about actors in general and then about Akka actors.

1.6 Actors: One programming model to rule up and out

Most general-purpose programming languages are written in sequence (Scala and Java are no exception to the rule). A concurrent programming model is required to bridge the gap between sequential definition and parallel execution.

Whereas parallelization is all about executing processes simultaneously, concurrency concerns itself with defining processes that *can* function simultaneously or overlap in time but don't necessarily *need* to run simultaneously. A concurrent system is not, by definition, a parallel system. Concurrent processes can, for